

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 8 — Tree-Based Methods Lab

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

1 Introduction

A decision tree predicts an outcome by repeatedly splitting the predictor space into simpler regions. The result is easy to explain: observations that follow the same sequence of splits receive the same prediction. Trees are therefore attractive when interpretability, nonlinear relationships, and interactions matter.

Their main weakness is instability. A small change in the training data can change an early split, which can change the entire tree below it. This lab starts with individual classification and regression trees, then introduces three ensemble approaches that address this instability in different ways:

1. pruning controls the complexity of one tree;
2. Bagging averages many trees trained on bootstrap samples;
3. Random Forests add random feature selection to reduce correlation between the trees; and
4. Boosting builds trees sequentially to correct the current model's errors.

The lab follows the Chapter 8.3 workflow from *An Introduction to Statistical Learning*. The explanations state the practical idea first and then connect it to the relevant statistical objective.

```
# Install these once if necessary:
# install.packages(c("ISLR2", "tree", "randomForest", "gbm", "BART"))

library(ISLR2)
library(tree)
library(randomForest)
library(gbm)
library(BART)
```

2 Classification Trees

2.1 The Carseats data

The `Carseats` data describe the sales of child car seats at 400 stores. Our first task is classification: we create a binary response indicating whether sales are high or low.

A store is labelled `Yes` when `Sales` exceeds 8 thousand units and `No` otherwise. This threshold is a modelling choice: it changes a continuous business outcome into a decision problem that could support an action such as identifying stores for extra promotion.

```
data(Carseats)
High <- factor(ifelse(Carseats$Sales <= 8, "No", "Yes"))
Carseats <- data.frame(Carseats, High)

str(Carseats)

## 'data.frame':    400 obs. of  12 variables:
##  $ Sales      : num  9.5 11.22 10.06 7.4 4.15 ...
##  $ CompPrice  : num  138 111 113 117 141 124 115 136 132 132 ...
##  $ Income     : num  73 48 35 100 64 113 105 81 110 113 ...
##  $ Advertising: num  11 16 10 4 3 13 0 15 0 0 ...
##  $ Population : num  276 260 269 466 340 501 45 425 108 131 ...
```

```
## $ Price      : num  120 83 80 97 128 72 108 120 124 124 ...
## $ ShelfLoc   : Factor w/ 3 levels "Bad","Good","Medium": 1 2 3 3 1 1 3 2 3 3 ...
## $ Age        : num   42 65 59 55 38 78 71 67 76 76 ...
## $ Education  : num   17 10 12 14 13 16 15 10 10 17 ...
## $ Urban      : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 1 2 2 1 1 ...
## $ US         : Factor w/ 2 levels "No","Yes": 2 2 2 2 1 2 1 2 1 2 ...
## $ High       : Factor w/ 2 levels "No","Yes": 2 2 2 1 1 2 1 2 1 1 ...
```

```
table(Carseats$High)
```

```
##
## No Yes
## 236 164
```

2.2 Growing a classification tree

A classification tree partitions the predictor space into regions and assigns one class to each terminal region. At a candidate split, the algorithm looks for a partition that makes the resulting child nodes purer. One common purity measure is the Gini index:

$$G = \sum_{k=1}^K \hat{p}_k(1 - \hat{p}_k),$$

where $\hat{p}_k$ is the proportion of observations in the node belonging to class k . A node containing only one class has Gini index zero; a mixed node has a larger value.

The `tree()` function grows a large tree first. This is useful because the subsequent pruning step can compare smaller subtrees obtained from the large candidate tree.

```
tree.carseats <- tree(High ~ . - Sales, data = Carseats)
tree.carseats
```

```
## node), split, n, deviance, yval, (yprob)
##      * denotes terminal node
##
##  1) root 400 541.500 No ( 0.59000 0.41000 )
##    2) ShelfLoc: Bad,Medium 315 390.600 No ( 0.68889 0.31111 )
##      4) Price < 92.5 46 56.530 Yes ( 0.30435 0.69565 )
##        8) Income < 57 10 12.220 No ( 0.70000 0.30000 )
##          16) CompPrice < 110.5 5 0.000 No ( 1.00000 0.00000 ) *
##          17) CompPrice > 110.5 5 6.730 Yes ( 0.40000 0.60000 ) *
##          9) Income > 57 36 35.470 Yes ( 0.19444 0.80556 )
##            18) Population < 207.5 16 21.170 Yes ( 0.37500 0.62500 ) *
##            19) Population > 207.5 20 7.941 Yes ( 0.05000 0.95000 ) *
##        5) Price > 92.5 269 299.800 No ( 0.75465 0.24535 )
##          10) Advertising < 13.5 224 213.200 No ( 0.81696 0.18304 )
##            20) CompPrice < 124.5 96 44.890 No ( 0.93750 0.06250 )
##              40) Price < 106.5 38 33.150 No ( 0.84211 0.15789 )
##                80) Population < 177 12 16.300 No ( 0.58333 0.41667 )
##                  160) Income < 60.5 6 0.000 No ( 1.00000 0.00000 ) *
```

```

##          161) Income > 60.5 6    5.407 Yes ( 0.16667 0.83333 ) *
##          81) Population > 177 26    8.477 No ( 0.96154 0.03846 ) *
##          41) Price > 106.5 58    0.000 No ( 1.00000 0.00000 ) *
##          21) CompPrice > 124.5 128 150.200 No ( 0.72656 0.27344 )
##          42) Price < 122.5 51    70.680 Yes ( 0.49020 0.50980 )
##          84) ShelveLoc: Bad 11    6.702 No ( 0.90909 0.09091 ) *
##          85) ShelveLoc: Medium 40  52.930 Yes ( 0.37500 0.62500 )
##          170) Price < 109.5 16    7.481 Yes ( 0.06250 0.93750 ) *
##          171) Price > 109.5 24    32.600 No ( 0.58333 0.41667 )
##          342) Age < 49.5 13    16.050 Yes ( 0.30769 0.69231 ) *
##          343) Age > 49.5 11    6.702 No ( 0.90909 0.09091 ) *
##          43) Price > 122.5 77    55.540 No ( 0.88312 0.11688 )
##          86) CompPrice < 147.5 58    17.400 No ( 0.96552 0.03448 ) *
##          87) CompPrice > 147.5 19    25.010 No ( 0.63158 0.36842 )
##          174) Price < 147 12    16.300 Yes ( 0.41667 0.58333 )
##          348) CompPrice < 152.5 7    5.742 Yes ( 0.14286 0.85714 ) *
##          349) CompPrice > 152.5 5    5.004 No ( 0.80000 0.20000 ) *
##          175) Price > 147 7    0.000 No ( 1.00000 0.00000 ) *
##          11) Advertising > 13.5 45    61.830 Yes ( 0.44444 0.55556 )
##          22) Age < 54.5 25    25.020 Yes ( 0.20000 0.80000 )
##          44) CompPrice < 130.5 14    18.250 Yes ( 0.35714 0.64286 )
##          88) Income < 100 9    12.370 No ( 0.55556 0.44444 ) *
##          89) Income > 100 5    0.000 Yes ( 0.00000 1.00000 ) *
##          45) CompPrice > 130.5 11    0.000 Yes ( 0.00000 1.00000 ) *
##          23) Age > 54.5 20    22.490 No ( 0.75000 0.25000 )
##          46) CompPrice < 122.5 10    0.000 No ( 1.00000 0.00000 ) *
##          47) CompPrice > 122.5 10    13.860 No ( 0.50000 0.50000 )
##          94) Price < 125 5    0.000 Yes ( 0.00000 1.00000 ) *
##          95) Price > 125 5    0.000 No ( 1.00000 0.00000 ) *
##          3) ShelveLoc: Good 85    90.330 Yes ( 0.22353 0.77647 )
##          6) Price < 135 68    49.260 Yes ( 0.11765 0.88235 )
##          12) US: No 17    22.070 Yes ( 0.35294 0.64706 )
##          24) Price < 109 8    0.000 Yes ( 0.00000 1.00000 ) *
##          25) Price > 109 9    11.460 No ( 0.66667 0.33333 ) *
##          13) US: Yes 51    16.880 Yes ( 0.03922 0.96078 ) *
##          7) Price > 135 17    22.070 No ( 0.64706 0.35294 )
##          14) Income < 46 6    0.000 No ( 1.00000 0.00000 ) *
##          15) Income > 46 11    15.160 Yes ( 0.45455 0.54545 ) *

```

```

cat("Terminal nodes:", sum(tree.carseats$frame$var == "<leaf>"), "\n")

```

```

## Terminal nodes: 27

```

```

cat("Training misclassification rate:",
    mean(predict(tree.carseats, type = "class") != Carseats$High), "\n")

```

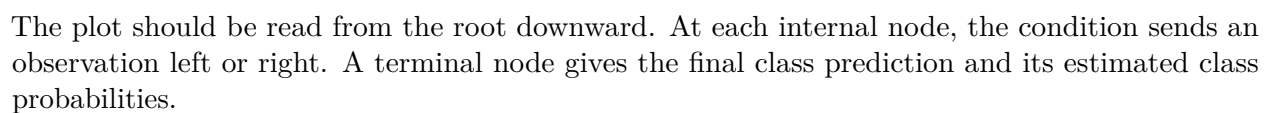
```

## Training misclassification rate: 0.09

```

The summary reports the number of terminal nodes and the training error. A very large tree can have a low training error because it has enough regions to memorise the training sample. That is

```
plot(tree.carseats)
text(tree.carseats, pretty = 0)
```



A tree with too many terminal nodes has high variance, while a tree that is too small may have high bias. Cost-complexity pruning makes this tradeoff explicit. For a subtree T , a common objective is

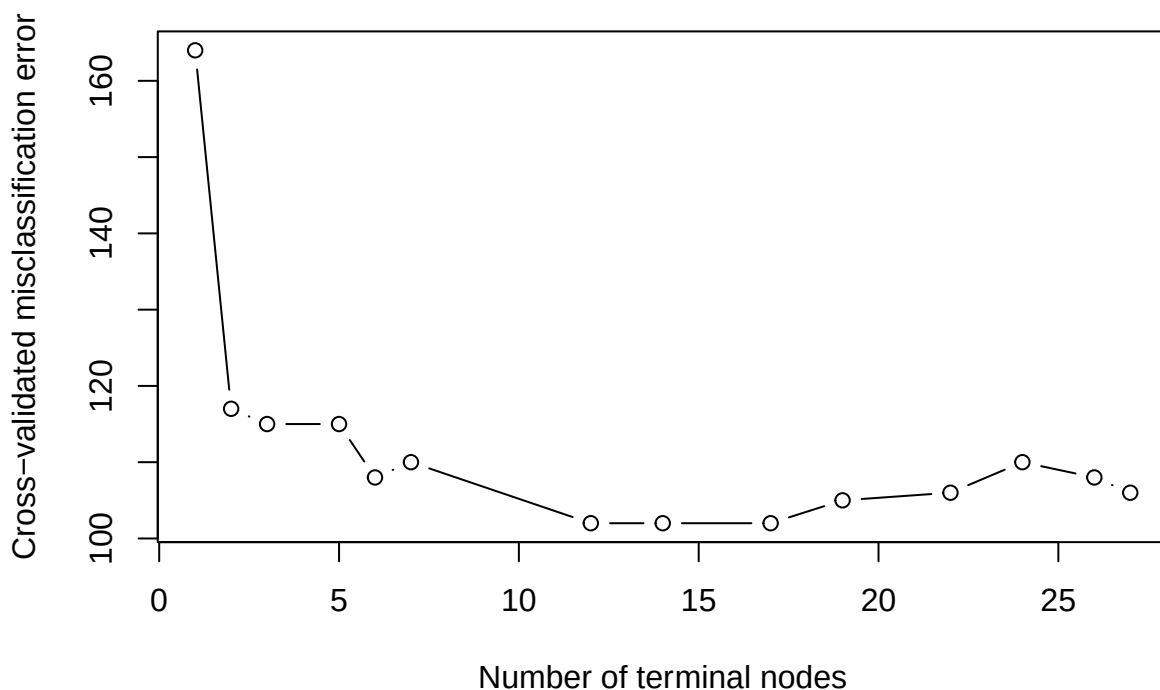
where $R(T)$ measures the tree's training misclassification error, $|T|$ is the number of terminal nodes, and $\alpha \geq 0$ penalizes complexity. A larger α favours a smaller tree.

```
set.seed(1)
cv.carseats <- cv.tree(tree.carseats, FUN = prune.misclass)
cv.carseats
```

```
## $size
## [1] 27 26 24 22 19 17 14 12 7 6 5 3 2 1
##
## $dev
## [1] 106 108 110 106 105 102 102 102 110 108 115 115 117 164
##
## $k
## [1] -Inf 0.000000 0.500000 1.000000 1.333333 1.500000 1.666667
## [8] 2.500000 3.800000 4.000000 5.000000 7.500000 18.000000 47.000000
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune" "tree.sequence"
```

```
plot(cv.carseats$size, cv.carseats$dev, type = "b",
     xlab = "Number of terminal nodes",
     ylab = "Cross-validated misclassification error",
     main = "Choosing the classification-tree size")
```

Choosing the classification-tree size



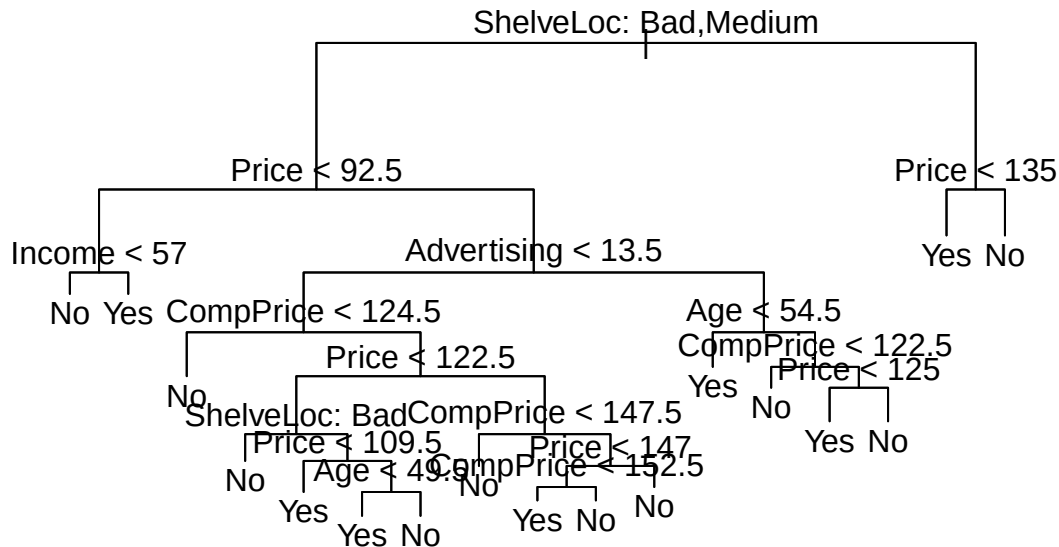
The lowest point is the cross-validation choice. Because cross-validation is random, a different seed can move the minimum slightly; the important idea is that tree size is selected using held-out predictions rather than training error alone.

```
prune.carseats <- prune.misclass(tree.carseats,
                                best = cv.carseats$size[which.min(cv.carseats$dev)])
```

```
summary(prune.carseats)
```

```
##
## Classification tree:
## snip.tree(tree = tree.carseats, nodes = c(9L, 22L, 7L, 8L, 20L,
## 6L))
## Variables actually used in tree construction:
## [1] "ShelveLoc" "Price" "Income" "Advertising" "CompPrice"
## [6] "Age"
## Number of terminal nodes: 17
## Residual mean deviance: 0.6632 = 254 / 383
## Misclassification error rate: 0.115 = 46 / 400
```

```
plot(prune.carseats)
text(prune.carseats, pretty = 0)
```



Pruning usually produces a simpler tree. Simplicity is useful for interpretation, but it is not automatically optimal; the cross-validation estimate is the evidence used to choose the amount of pruning.

3 Regression Trees

3.1 The Hitters data

We now predict a quantitative response: baseball players' salaries. The regression-tree analogue of a classification tree assigns a constant prediction to every terminal node, usually the mean response of the training observations in that node.

A regression tree chooses splits to reduce residual sum of squares (RSS):

$$\text{RSS}(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2,$$

where R_m is a terminal region and $\hat{y}_{R_m}$ is the mean response in that region. Again, an extremely large tree can reduce training RSS while performing poorly on new observations.

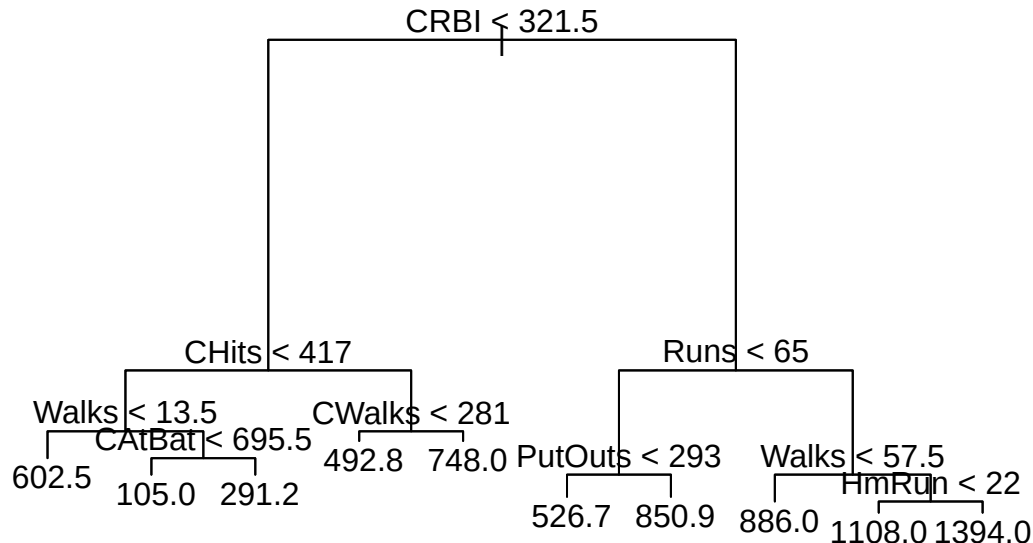
```
data(Hitters)
Hitters <- na.omit(Hitters)

set.seed(1)
train <- sample(1:nrow(Hitters), nrow(Hitters) / 2)
Hitters.train <- Hitters[train, ]
Hitters.test <- Hitters[-train, ]

tree.hitters <- tree(Salary ~ ., data = Hitters.train)
summary(tree.hitters)

##
## Regression tree:
## tree(formula = Salary ~ ., data = Hitters.train)
## Variables actually used in tree construction:
## [1] "CRBI"      "CHits"     "Walks"     "CAtBat"    "CWalks"    "Runs"      "PutOuts"
## [8] "HmRun"
## Number of terminal nodes: 10
## Residual mean deviance: 58340 = 7060000 / 121
## Distribution of residuals:
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -450.90 -103.70  -16.21     0.00  74.54 1525.00

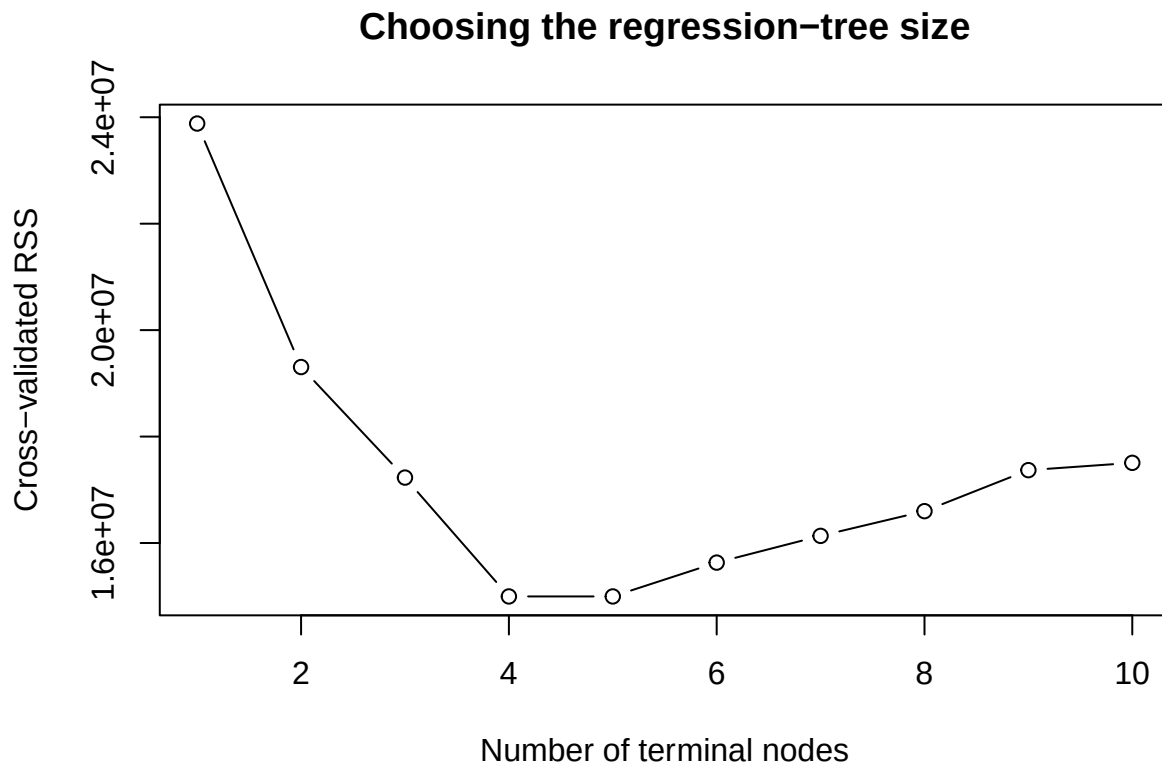
plot(tree.hitters)
text(tree.hitters, pretty = 0)
```



The first split is the predictor that gives the largest immediate reduction in RSS under the algorithm's greedy search. Later splits represent interactions: their meaning is conditional on the decisions made higher in the tree.

3.2 Cross-validation and a test-set check

```
set.seed(1)
cv.hitters <- cv.tree(tree.hitters)
plot(cv.hitters$size, cv.hitters$dev, type = "b",
     xlab = "Number of terminal nodes",
     ylab = "Cross-validated RSS",
     main = "Choosing the regression-tree size")
```



```
prune.hitters <- prune.tree(tree.hitters,
                           best = cv.hitters$size[which.min(cv.hitters$dev)])
summary(prune.hitters)
```

```
##
## Regression tree:
## snip.tree(tree = tree.hitters, nodes = c(5L, 15L, 6L, 4L))
## Variables actually used in tree construction:
## [1] "CRBI" "CHits" "Runs" "Walks"
## Number of terminal nodes: 5
## Residual mean deviance: 74730 = 9416000 / 126
## Distribution of residuals:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -451.90 -153.10  -41.82    0.00  100.70  1891.00
```

We evaluate the unpruned and pruned models on the same test set. Test error is an estimate of future prediction error because these observations were not used to fit the tree.

```

pred.tree <- predict(tree.hitters, Hitters.test)
pred.pruned <- predict(prune.hitters, Hitters.test)

mse.tree <- mean((pred.tree - Hitters.test$Salary)^2)
mse.pruned <- mean((pred.pruned - Hitters.test$Salary)^2)

c(unpruned_MSE = mse.tree, pruned_MSE = mse.pruned)

## unpruned_MSE    pruned_MSE
##      122872.5      122267.2

```

The better model is the one with the smaller test MSE in this split. The comparison also illustrates an important qualification: pruning can improve generalisation, but the result depends on the sample split and on the signal in the data. The purpose of the test set is to assess the final choice, not to choose repeatedly until a favourable result appears.

4 Bagging and Random Forests

4.1 Why averaging trees helps

Bagging means bootstrap aggregating. We repeatedly sample n observations with replacement, grow a tree on each bootstrap sample, and average the predictions. For regression,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

If the trees have variance σ^2 and pairwise correlation ρ , then under the usual simplifying assumptions,

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B T_b(x) \right) = \sigma^2 \left(\rho + \frac{1-\rho}{B} \right).$$

Increasing B reduces the unshared part of the variance. It cannot remove error shared by all trees, which is why reducing tree correlation is valuable.

4.2 Bagging on Boston

The textbook uses the `Boston` data to compare tree ensembles for predicting `medv`, the median value of owner-occupied homes in a Boston suburb. We use a fixed train/test split so the models are compared on exactly the same cases.

```

Boston <- ISLR2::Boston
set.seed(1)
train <- sample(1:nrow(Boston), nrow(Boston) / 2)
Boston.train <- Boston[train, ]
Boston.test <- Boston[-train, ]

```

For bagging, `mtry = 13` allows each split to consider all predictors. The bootstrap samples create the variation between trees.

```

set.seed(1)
bag.boston <- randomForest(
  medv ~ ., data = Boston.train,
  mtry = ncol(Boston.train) - 1,
  importance = TRUE
)

bag.pred <- predict(bag.boston, Boston.test)
bag.mse <- mean((bag.pred - Boston.test$medv)^2)
bag.mse

```

```
## [1] 23.41916
```

The out-of-bag estimate is produced by predicting each training observation using only trees whose bootstrap sample did not contain that observation. It provides an internal performance estimate without a separate validation set.

```
bag.boston$mse[length(bag.boston$mse)]
```

```
## [1] 11.40162
```

4.3 Random Forests

Random Forests add a second source of randomness. At each split, only a random subset of the predictors is considered. If the strongest predictor is not always available, different trees are forced to explore different structures, reducing their correlation ρ .

```

set.seed(1)
rf.boston <- randomForest(
  medv ~ ., data = Boston.train,
  mtry = 6,
  importance = TRUE
)

rf.pred <- predict(rf.boston, Boston.test)
rf.mse <- mean((rf.pred - Boston.test$medv)^2)
rf.mse

```

```
## [1] 20.06644
```

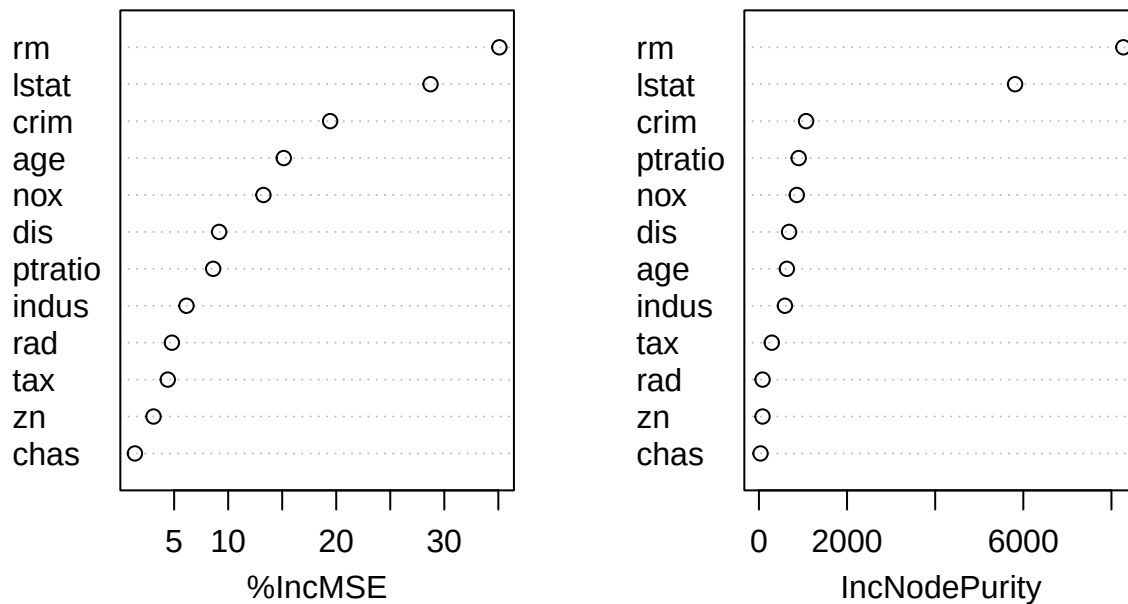
```
importance(rf.boston)
```

```
##           %IncMSE IncNodePurity
## crim    19.435587    1070.42307
## zn       3.091630      82.19257
## indus    6.140529     590.09536
## chas     1.370310      36.70356
## nox     13.263466     859.97091
## rm      35.094741    8270.33906
## age     15.144821     634.31220
## dis      9.163776     684.87953
```

```
## rad      4.793720      83.18719
## tax      4.410714     292.20949
## ptratio  8.612780     902.20190
## lstat    28.725343    5813.04833
```

```
varImpPlot(rf.boston)
```

rf.boston



The test MSEs provide an empirical comparison of bagging and the Random Forest. The Random Forest is not guaranteed to win on every split: its design trades a little individual-tree strength for lower correlation, and the tradeoff is data-dependent.

```
data.frame(
  Method = c("Bagging", "Random Forest"),
  Test_MSE = c(bag.mse, rf.mse)
)
```

```
##      Method Test_MSE
## 1      Bagging 23.41916
## 2 Random Forest 20.06644
```

5 Boosting

5.1 Sequential error correction

Boosting does not grow independent trees and average them. Instead, it builds an additive model one tree at a time:

$$F_m(x) = F_{m-1}(x) + \lambda h_m(x),$$

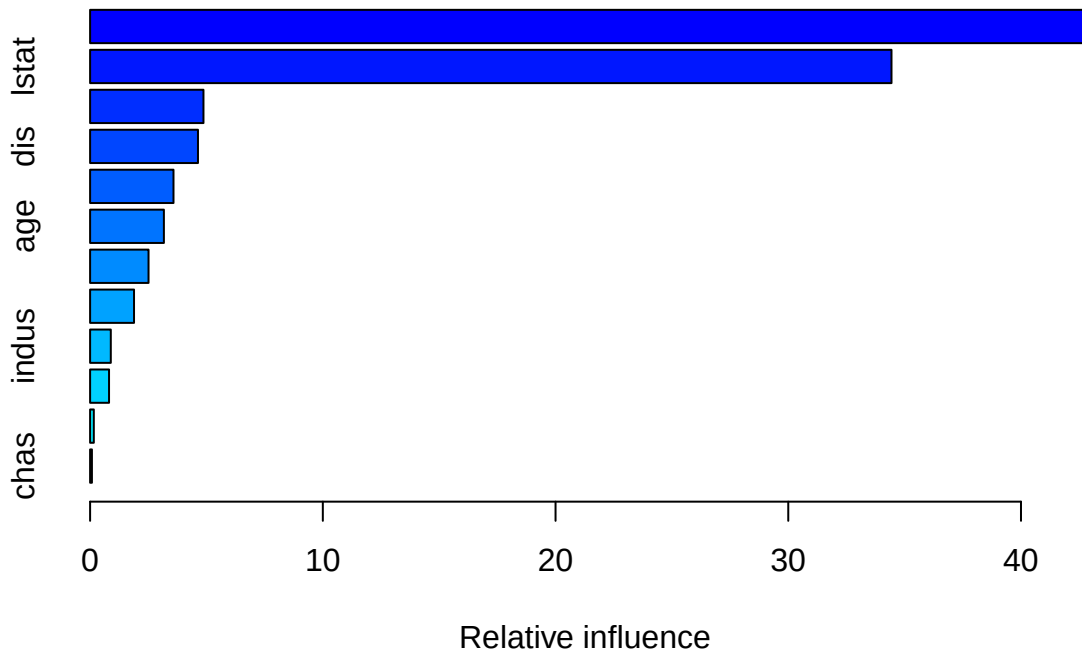
where h_m is a new, usually shallow, tree and λ is the learning rate. For squared-error regression, the next tree is fitted to the current residuals $y_i - F_{m-1}(x_i)$. The new tree is therefore a correction, not a complete model trained from scratch.

5.2 Gradient boosting on Boston

The `gbm()` implementation uses stochastic gradient boosting. Its important controls are the number of trees, interaction depth, and shrinkage (learning rate). A smaller learning rate usually requires more trees but makes the updates more gradual.

```
set.seed(1)
boost.boston <- gbm(
  medv ~ ., data = Boston.train,
  distribution = "gaussian",
  n.trees = 5000,
  interaction.depth = 4,
  shrinkage = 0.01,
  verbose = FALSE
)

summary(boost.boston)
```

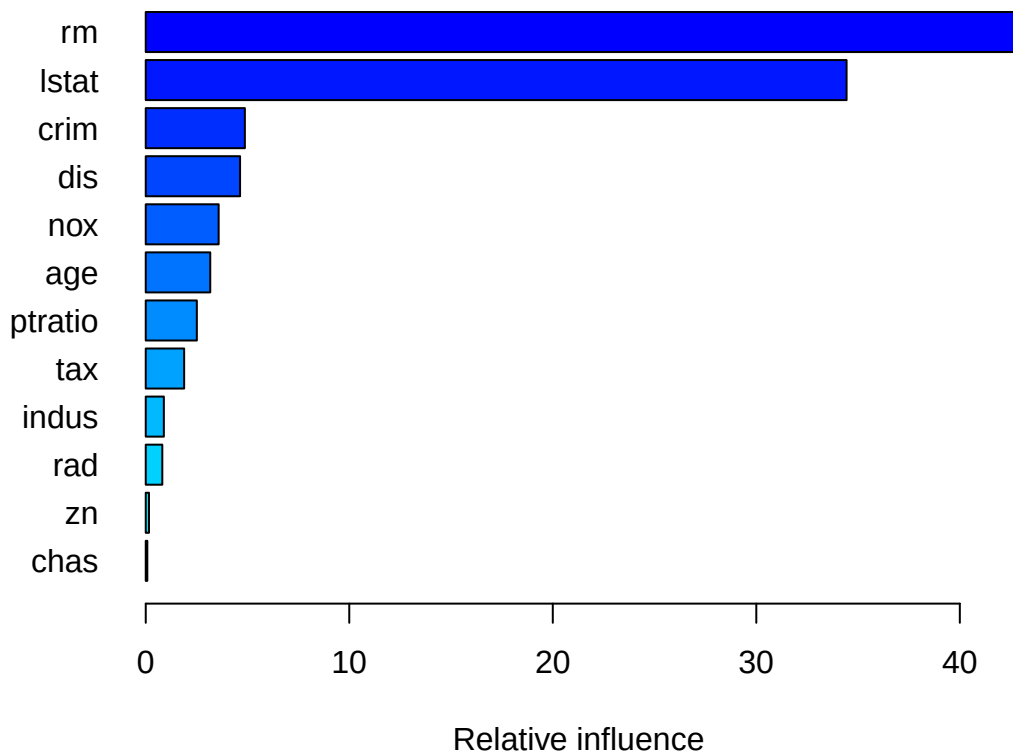


```
##          var      rel.inf
## rm          rm 42.96709666
## lstat      lstat 34.43613309
## crim       crim  4.87078782
## dis        dis  4.63504360
```

```
## nox      nox  3.58276509
## age      age  3.17018795
## ptratio ptratio 2.51023644
## tax      tax  1.88741011
## indus    indus 0.88953756
## rad      rad  0.81391326
## zn       zn   0.15908135
## chas     chas 0.07780706
```

The variable-importance plot summarizes how much each variable contributes to reducing the loss across the boosted trees. It is a useful descriptive tool, not proof that a variable causes the response.

```
par(mar = c(5, 10, 4, 2))
summary(boost.boston, las = 1)
```



```
##      var      rel.inf
## rm      rm 42.96709666
## lstat    lstat 34.43613309
## crim     crim  4.87078782
## dis      dis  4.63504360
## nox      nox  3.58276509
## age      age  3.17018795
## ptratio ptratio 2.51023644
## tax      tax  1.88741011
## indus    indus 0.88953756
## rad      rad  0.81391326
## zn       zn   0.15908135
## chas     chas 0.07780706
```

```
boost.pred <- predict(boost.boston, Boston.test, n.trees = 5000)
boost.mse <- mean((boost.pred - Boston.test$medv)^2)
boost.mse
```

```
## [1] 17.24452
```

```
ensemble.results <- data.frame(
  Method = c("Bagging", "Random Forest", "Boosting"),
  Test_MSE = c(bag.mse, rf.mse, boost.mse)
)
ensemble.results[order(ensemble.results$Test_MSE), ]
```

```
##           Method Test_MSE
## 3      Boosting 17.24452
## 2 Random Forest 20.06644
## 1      Bagging 23.41916
```

The smallest test MSE in this table is the best result for this particular split and set of tuning choices. It should not be interpreted as a universal ranking. In practice, the number of trees, depth, learning rate, and feature sampling settings should be selected using cross-validation or a validation set, and the final test set should be used only once for the final assessment.

6 Bayesian Additive Regression Trees

Bayesian Additive Regression Trees (BART) also represent the response as a sum of many trees:

$$Y = \sum_{j=1}^m g(X; T_j, M_j) + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2).$$

Here T_j is the structure of tree j and M_j contains its terminal-node values. Unlike ordinary boosting, BART puts prior distributions on tree structures and leaf parameters, then uses posterior sampling to represent uncertainty about the ensemble.

```
set.seed(1)
x.train <- as.matrix(Boston.train[, setdiff(names(Boston.train), "medv")])
y.train <- Boston.train$medv
x.test <- as.matrix(Boston.test[, setdiff(names(Boston.test), "medv")])

bart.fit <- wbart(
  x.train, y.train,
  x.test = x.test,
  nskip = 1000,
  ndpost = 2000,
  ntree = 200
)
```

```
## *****Into main of wbart
## *****Data:
## data:n,p,np: 253, 12, 253
```

```

## y1,yn: 0.213439, -5.486561
## x1,x[n*p]: 0.109590, 20.080000
## xp1,xp[np*p]: 0.027310, 7.880000
## *****Number of Trees: 200
## *****Number of Cut Points: 100 ... 100
## *****burn and ndpost: 1000, 2000
## *****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.795495,3.000000,3.716363
## *****sigma: 4.367914
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,12,0
## *****nkeeptrain,nkeepptest,nkeepstme,nkeepreedraws: 2000,2000,2000,2000
## *****printevery: 100
## *****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1
##
## MCMC
## done 0 (out of 3000)
## done 100 (out of 3000)
## done 200 (out of 3000)
## done 300 (out of 3000)
## done 400 (out of 3000)
## done 500 (out of 3000)
## done 600 (out of 3000)
## done 700 (out of 3000)
## done 800 (out of 3000)
## done 900 (out of 3000)
## done 1000 (out of 3000)
## done 1100 (out of 3000)
## done 1200 (out of 3000)
## done 1300 (out of 3000)
## done 1400 (out of 3000)
## done 1500 (out of 3000)
## done 1600 (out of 3000)
## done 1700 (out of 3000)
## done 1800 (out of 3000)
## done 1900 (out of 3000)
## done 2000 (out of 3000)
## done 2100 (out of 3000)
## done 2200 (out of 3000)
## done 2300 (out of 3000)
## done 2400 (out of 3000)
## done 2500 (out of 3000)
## done 2600 (out of 3000)
## done 2700 (out of 3000)
## done 2800 (out of 3000)
## done 2900 (out of 3000)
## time: 5s
## check counts
## trcnt,tecnt,temecnt,treedrawscnt: 2000,2000,2000,2000

```

```

bart.pred <- colMeans(bart.fit$yhat.test)
bart.mse <- mean((bart.pred - Boston.test$medv)^2)
bart.mse

```

```
## [1] 16.0127
```

The BART prediction is the posterior mean of the test predictions. Its posterior draws also contain uncertainty information, which is a major conceptual difference from a single fitted tree or a point-prediction-only ensemble.

```

final.results <- rbind(
  ensemble.results,
  data.frame(Method = "BART", Test_MSE = bart.mse)
)
final.results[order(final.results$Test_MSE), ]

```

```

##           Method Test_MSE
## 4           BART 16.01270
## 3      Boosting 17.24452
## 2 Random Forest 20.06644
## 1      Bagging 23.41916

```

7 Conclusion

A single tree gives an interpretable sequence of decisions, but its structure can be unstable. Pruning controls that instability by selecting a smaller subtree. Bagging reduces variance by averaging trees grown on bootstrap samples. Random Forests improve the averaging strategy by also reducing the correlation between trees through random feature selection. Boosting follows a different path: it grows trees sequentially, with each tree correcting the current ensemble. BART extends the additive-tree idea with Bayesian priors and posterior uncertainty.

The main lesson is not that one method always wins. The appropriate method depends on the response type, the signal-to-noise ratio, the amount of tuning available, and whether interpretability or predictive accuracy is the primary goal. A fair comparison requires the same data split or resampling scheme, careful control of tuning, and a final evaluation on observations not used to fit the chosen model.